

Python Dictionaries

Coding Assignment — 30 Questions

Topics Covered	Dictionary methods — .get() .pop() .keys() .values() .items() .update() .copy() .clear() .setdefault() dict.fromkeys()
Difficulty	10 Basic · 10 Easy · 10 Medium
Constraint	All questions solvable using dictionary methods only — no loops, no conditionals

Section 1 — Basic Q1 – Q10

Each question uses a single dictionary method. Focus on getting comfortable with the syntax before combining methods.

Q1 Access a value by key	
BASIC	Method: <code>dict[]</code>
You have a student record stored as a dictionary. Retrieve the student's age directly using its key.	
INPUT / SETUP <pre>student = {"name": "Ali", "age": 20, "grade": "A"}</pre>	EXPECTED OUTPUT 20

Q2 Safely get a value with a default	
BASIC	Method: <code>.get()</code>
A product dictionary may or may not have a 'discount' key. Use <code>.get()</code> to retrieve it safely and return 0 if it does not exist.	
INPUT / SETUP <pre>product = {"name": "Laptop", "price": 999}</pre>	EXPECTED OUTPUT 0

Q3 Add a new key-value pair

BASIC

Method: dict[] =

A user profile is missing an 'email' field. Add it to the dictionary.

INPUT / SETUP

```
profile = {"username": "sahil",  
"age": 25}
```

EXPECTED OUTPUT

```
{"username": "sahil", "age": 25,  
"email": "sahil@email.com"}
```

Q4 Update an existing value

BASIC

Method: dict[] =

A student's grade has been revised. Update the 'grade' key in the dictionary to the new value 'A+'.

INPUT / SETUP

```
student = {"name": "Mia", "grade":  
"B"}
```

EXPECTED OUTPUT

```
{"name": "Mia", "grade": "A+"}
```

Q5 Remove a key and get its value

BASIC

Method: .pop()

Remove the 'temp' key from a session dictionary and store its value in a variable.

INPUT / SETUP

```
session = {"user": "Ali", "temp":  
"xyz123", "role": "admin"}
```

EXPECTED OUTPUT

```
removed = "xyz123"  
dict = {"user": "Ali", "role":  
"admin"}
```

Q6 Count the number of keys

BASIC	Method: len()
Find out how many fields are stored in a contact dictionary.	
INPUT / SETUP <pre>contact = {"name": "Leo", "phone": "555-1234", "city": "Toronto", "email": "leo@mail.com"}</pre>	EXPECTED OUTPUT <pre>4</pre>

Q7 Get all keys from a dictionary	
BASIC	Method: .keys()
Retrieve all the field names (keys) from an employee record.	
INPUT / SETUP <pre>employee = {"id": 101, "name": "Sara", "dept": "HR", "salary": 55000}</pre>	EXPECTED OUTPUT <pre>dict_keys(["id", "name", "dept", "salary"])</pre>

Q8 Get all values from a dictionary	
BASIC	Method: .values()
Retrieve all the values stored in a config dictionary.	
INPUT / SETUP <pre>config = {"host": "localhost", "port": 8080, "debug": True}</pre>	EXPECTED OUTPUT <pre>dict_values(["localhost", 8080, True])</pre>

Q9 Check if a key exists	
BASIC	Method: in

Before accessing a key, check whether 'phone' exists in the contact dictionary.

INPUT / SETUP

```
contact = {"name": "Eva", "email":  
"eva@mail.com"}
```

EXPECTED OUTPUT

```
False
```

Q10 Empty a dictionary

BASIC

Method: `.clear()`

A temporary cache dictionary needs to be wiped clean. Clear all its contents.

INPUT / SETUP

```
cache = {"a": 1, "b": 2, "c": 3}
```

EXPECTED OUTPUT

```
{}
```

Section 2 — Easy Q11 – Q20

These questions introduce slightly less obvious methods and require choosing the right tool for the task.

Q11 Get all key-value pairs

EASY

Method: `.items()`

Retrieve all the key-value pairs from an inventory dictionary as a view object.

INPUT / SETUP

```
inventory = {"apples": 50, "bananas":  
30, "mangoes": 20}
```

EXPECTED OUTPUT

```
dict_items([("apples", 50),  
("bananas", 30), ("mangoes", 20)])
```

Q12 Merge another dictionary into this one

EASY

Method: `.update()`

A student's record needs two new fields added at once. Use <code>.update()</code> to add both 'city' and 'phone' in one step.	
INPUT / SETUP <pre>student = {"name": "Ali", "age": 21} new_info = {"city": "Toronto", "phone": "555-0000"}</pre>	EXPECTED OUTPUT <pre>{"name": "Ali", "age": 21, "city": "Toronto", "phone": "555-0000"}</pre>

Q13 Overwrite values using update	
EASY	Method: <code>.update()</code>
A product's price and stock have changed. Use <code>.update()</code> to apply both changes at once.	
INPUT / SETUP <pre>product = {"name": "Phone", "price": 699, "stock": 100} updates = {"price": 649, "stock": 80}</pre>	EXPECTED OUTPUT <pre>{"name": "Phone", "price": 649, "stock": 80}</pre>

Q14 Get a value with a custom default message	
EASY	Method: <code>.get()</code>
Look up a student's score. If the subject does not exist, return 'Not enrolled' instead of an error.	
INPUT / SETUP <pre>scores = {"Math": 88, "Science": 75, "English": 90} subject = "History"</pre>	EXPECTED OUTPUT <pre>"Not enrolled"</pre>

Q15 Remove a key safely with a default	
EASY	Method: <code>.pop()</code>

Try to remove a 'nickname' key from a profile. If it does not exist, return 'No nickname' instead of raising an error.

INPUT / SETUP

```
profile = {"name": "Mia", "age": 22}
```

EXPECTED OUTPUT

```
"No nickname"
```

Q16 Create a dictionary from keyword arguments**EASY**

Method: `dict()`

Create a new dictionary for a book using the `dict()` constructor with keyword arguments.

INPUT / SETUP

```
title="Dune", author="Frank Herbert",  
year=1965
```

EXPECTED OUTPUT

```
{"title": "Dune", "author": "Frank  
Herbert", "year": 1965}
```

Q17 Make a shallow copy of a dictionary**EASY**

Method: `.copy()`

You want to modify a student's record without changing the original. First make a copy, then update the grade in the copy.

INPUT / SETUP

```
original = {"name": "Joe", "grade":  
"B"}
```

EXPECTED OUTPUT

```
copy = {"name": "Joe", "grade": "A"}  
original = {"name": "Joe", "grade":  
"B"}
```

Q18 Create a dictionary with default values**EASY**

Method: `dict.fromkeys()`

Create a dictionary for a new student where all subject scores start at 0.

INPUT / SETUP subjects = ["Math", "Science", "English", "History"]	EXPECTED OUTPUT { "Math": 0, "Science": 0, "English": 0, "History": 0 }
--	---

Q19 Check if a key is NOT in the dictionary	
EASY	Method: in + .keys()
Verify that 'salary' is not already present in an employee record before adding it.	
INPUT / SETUP employee = {"name": "Sara", "dept": "IT", "age": 30}	EXPECTED OUTPUT True (salary is not present)

Q20 Count total values and find their sum	
EASY	Method: len() + .values()
Find how many values are stored in the dictionary, then find the sum of all numeric values in a scores dictionary.	
INPUT / SETUP scores = {"Math": 88, "Science": 76, "English": 90, "History": 84}	EXPECTED OUTPUT Count: 4 Sum: 338

Section 3 — Medium Q21 – Q30

These questions chain two or more methods together in realistic scenarios like renaming keys, merging records, and finding min/max values.

Q21 Build a nested dictionary and access deep values	
MEDIUM	Method: dict[] + .get()

Create a nested dictionary for a school with two students, each having a name and score. Then retrieve the score of the second student using chained key access.

INPUT / SETUP

```
school = {  
    "student1": {"name": "Ali",  
                "score": 88},  
    "student2": {"name": "Mia",  
                "score": 95}  
}
```

EXPECTED OUTPUT

95

Q22 Rename a key in a dictionary

MEDIUM

Method: `.update()` + `.pop()`

A dictionary has a key called 'fname'. Rename it to 'first_name' without losing its value. Use `.pop()` to remove the old key and `.update()` to add the new one.

INPUT / SETUP

```
person = {"fname": "Alice", "age":  
28, "city": "London"}
```

EXPECTED OUTPUT

```
{"first_name": "Alice", "age": 28,  
"city": "London"}
```

Q23 Create a template and fill it in

MEDIUM

Method: `dict.fromkeys()` + `.update()`

First create a student template with keys 'name', 'age', 'grade' all set to None. Then update it with actual values.

INPUT / SETUP

```
keys = ["name", "age", "grade"]  
values = {"name": "Leo", "age": 19,  
"grade": "A"}
```

EXPECTED OUTPUT

```
{"name": "Leo", "age": 19, "grade":  
"A"}
```

Q24 Implement a visit counter

MEDIUM	Method: <code>.get()</code> + assignment
A page visit tracker stores page names and visit counts. Increment the visit count for 'home' by 1 using <code>.get()</code> to safely handle the case where the page may not exist yet.	
INPUT / SETUP <pre>visits = {"about": 10, "contact": 5} page = "home"</pre>	EXPECTED OUTPUT <pre>{"about": 10, "contact": 5, "home": 1}</pre>

Q25 Swap keys and values	
MEDIUM	Method: <code>.items()</code> + <code>dict()</code>
Given a dictionary of country codes, create a new dictionary where the values become keys and the keys become values.	
INPUT / SETUP <pre>codes = {"IN": "India", "US": "USA", "CA": "Canada"}</pre>	EXPECTED OUTPUT <pre>{"India": "IN", "USA": "US", "Canada": "CA"}</pre>

Q26 Extract keys and values as separate lists	
MEDIUM	Method: <code>.keys()</code> + <code>.values()</code> + <code>list()</code>
Given a score dictionary, extract all subject names into one list and all scores into another list.	
INPUT / SETUP <pre>scores = {"Math": 88, "Science": 76, "English": 90}</pre>	EXPECTED OUTPUT <pre>subjects = ["Math", "Science", "English"] marks = [88, 76, 90]</pre>

Q27 Find the key with the highest value	
MEDIUM	Method: <code>.get()</code> + <code>max()</code>

Given a dictionary of student scores, find which student scored the highest using <code>max()</code> with <code>.get</code> as the key argument.	
INPUT / SETUP <pre>scores = {"Ali": 88, "Mia": 95, "Joe": 72, "Eva": 91}</pre>	EXPECTED OUTPUT <pre>"Mia"</pre>

Q28 Merge two dictionaries into a third	
MEDIUM	Method: <code>.update() + dict()</code>
Given two separate dictionaries for personal info and academic info, merge both into a single new dictionary without modifying either original.	
INPUT / SETUP <pre>personal = {"name": "Sam", "age": 20} academic = {"grade": "A", "score": 92}</pre>	EXPECTED OUTPUT <pre>{"name": "Sam", "age": 20, "grade": "A", "score": 92}</pre>

Q29 Add a key only if it does not exist	
MEDIUM	Method: <code>.setdefault()</code>
Use <code>.setdefault()</code> to add a 'loyalty_points' key only if it is not already present. If the key exists, leave it unchanged.	
INPUT / SETUP <pre>customer1 = {"name": "Ali", "tier": "Gold"} customer2 = {"name": "Mia", "tier": "Silver", "loyalty_points": 200}</pre>	EXPECTED OUTPUT <pre>customer1 = {"name": "Ali", "tier": "Gold", "loyalty_points": 0} customer2 = {"name": "Mia", "tier": "Silver", "loyalty_points": 200}</pre>

Q30 Find the subject with the lowest score	
MEDIUM	Method: <code>.items() + min()</code>

Given a dictionary of exam scores, find the subject in which the student performed the worst using `min()` on `.items()`.

INPUT / SETUP

```
scores = {"Math": 78, "Science": 65,  
"English": 88, "History": 59}
```

EXPECTED OUTPUT

```
"History"
```

Python Dictionary Methods Assignment · Good luck!